Chapter 6: Modular Programming-Cont

Mohamed M. Sabry Aly

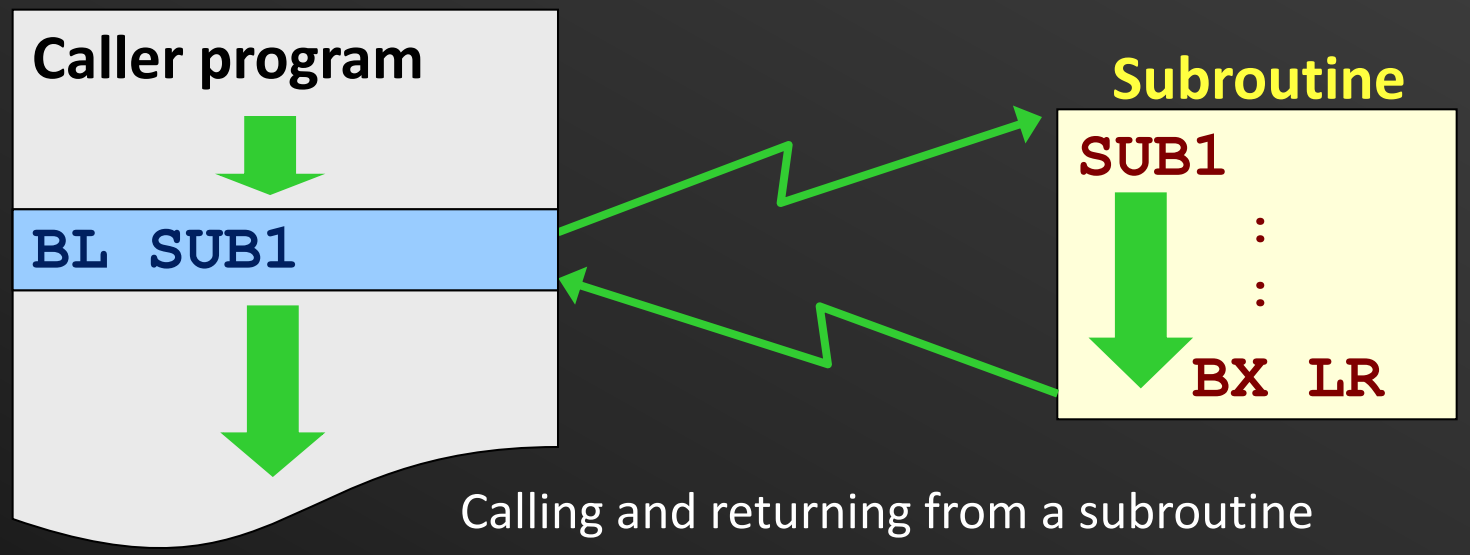
N4-02c-92

Learning Objectives

- Understand the concept of nested subroutine
- Describe how nested subroutines are implemented in ARM
- Describe recursive functions and their implementation in ARM

Subroutines (Recap)

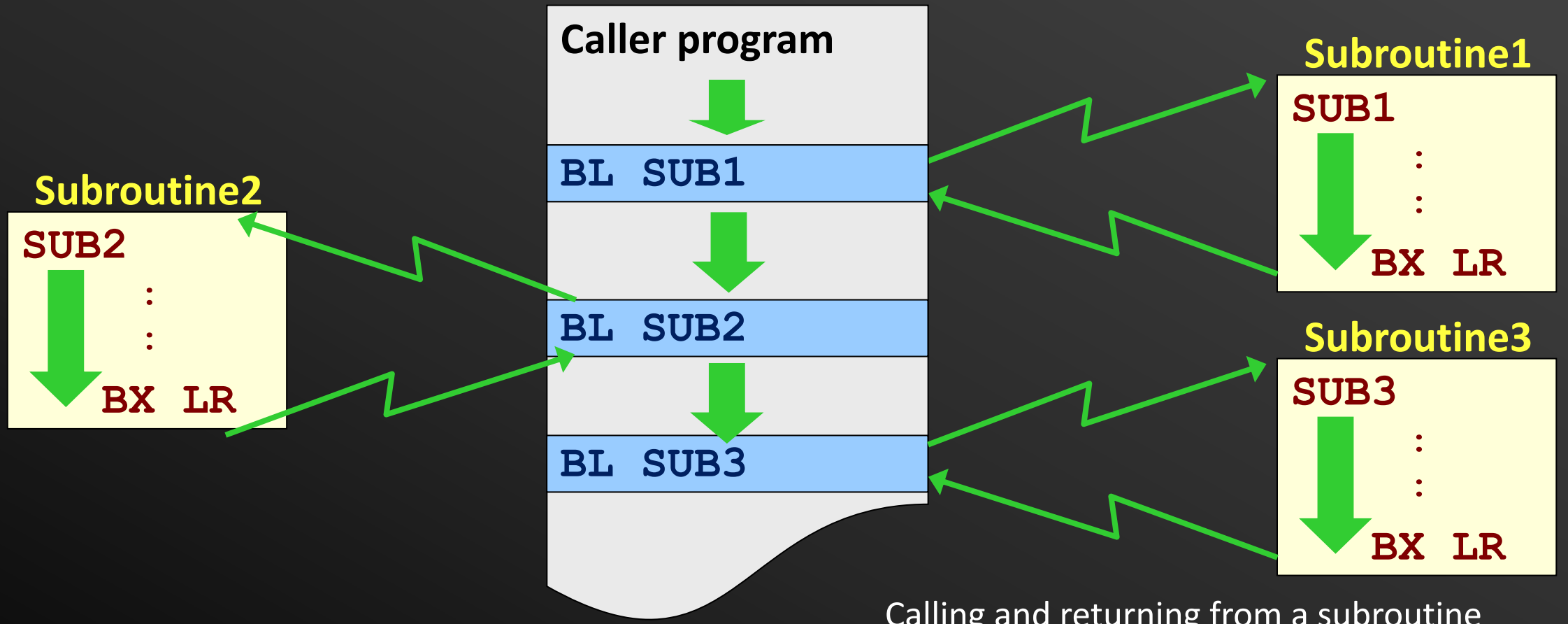
- Modules (e.g., functions in C) are implemented as **subroutines**
- Subroutine can be called from various parts of the program
- Caller and callee
  - Caller: the program that calls subroutine (SUB1)
  - Callee: subroutine (SUB1)



Calling and returning from a subroutine

Multiple Subroutines

- Caller program can call multiple subroutines
- Support of sequential subroutine execution

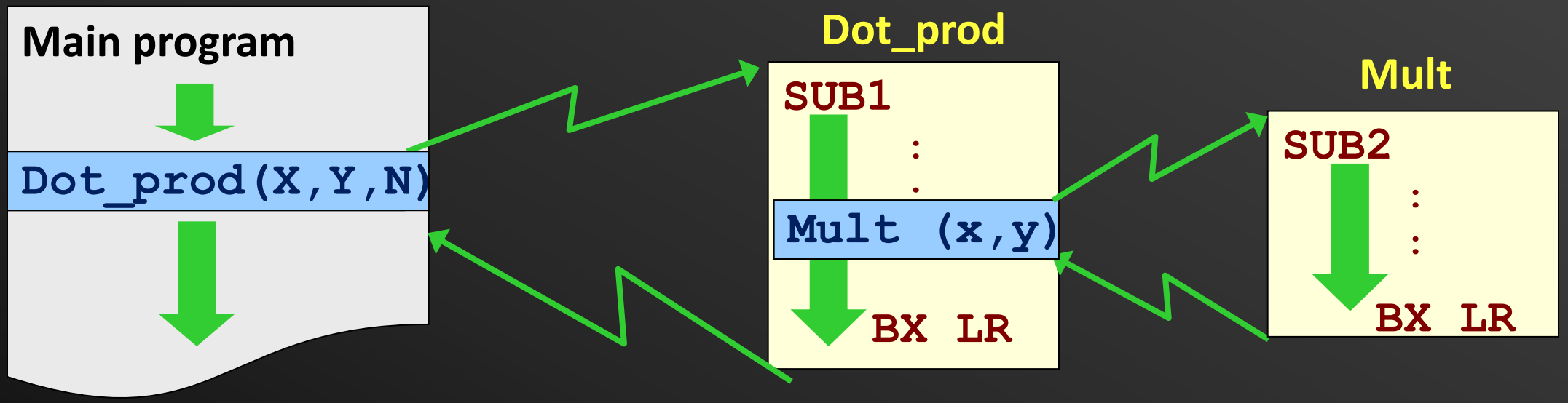


Calling and returning from a subroutine

Do all applications have just 1-level functional call?

Example: Dot product of two arrays

- A subroutine will call another subroutine



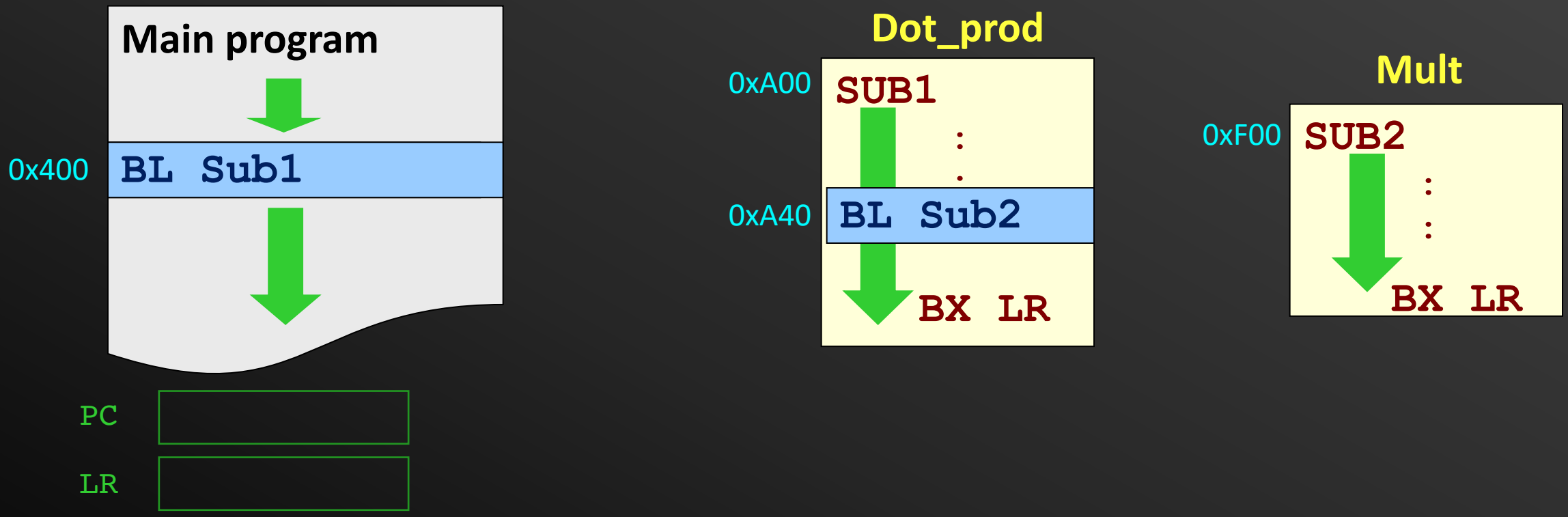
Example: Dot product of two arrays

- A subroutine will call another subroutine

How to ensure right subroutine branch and return?

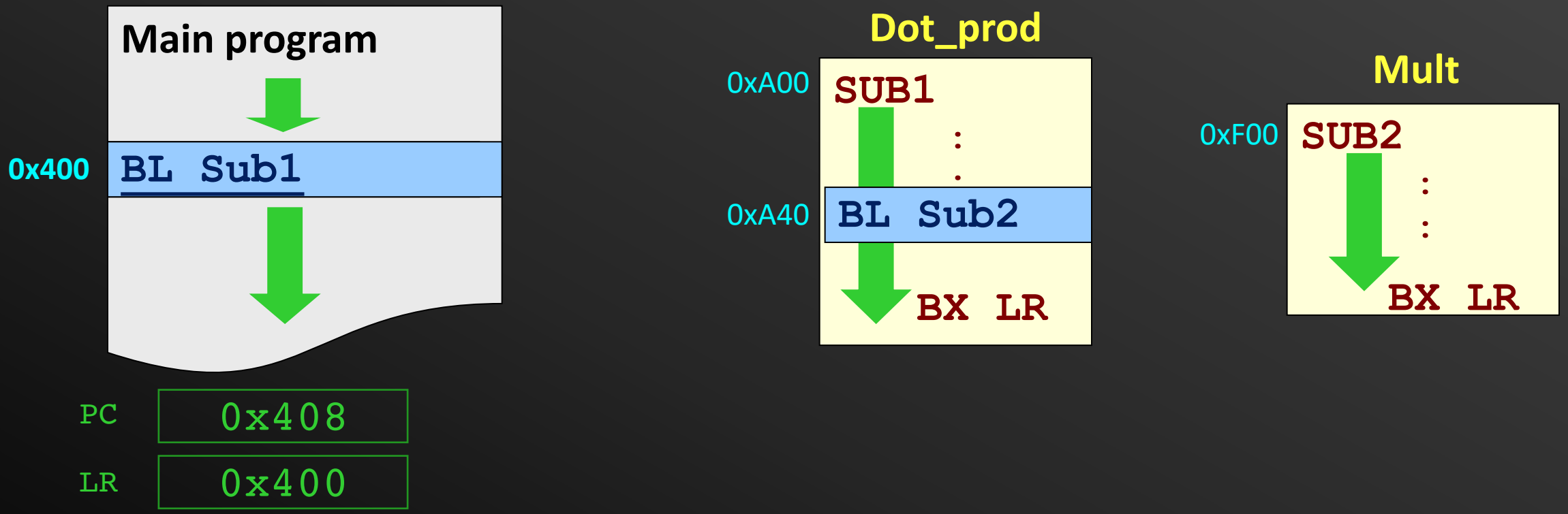
Example: Dot product of two arrays

- A subroutine will call another subroutine



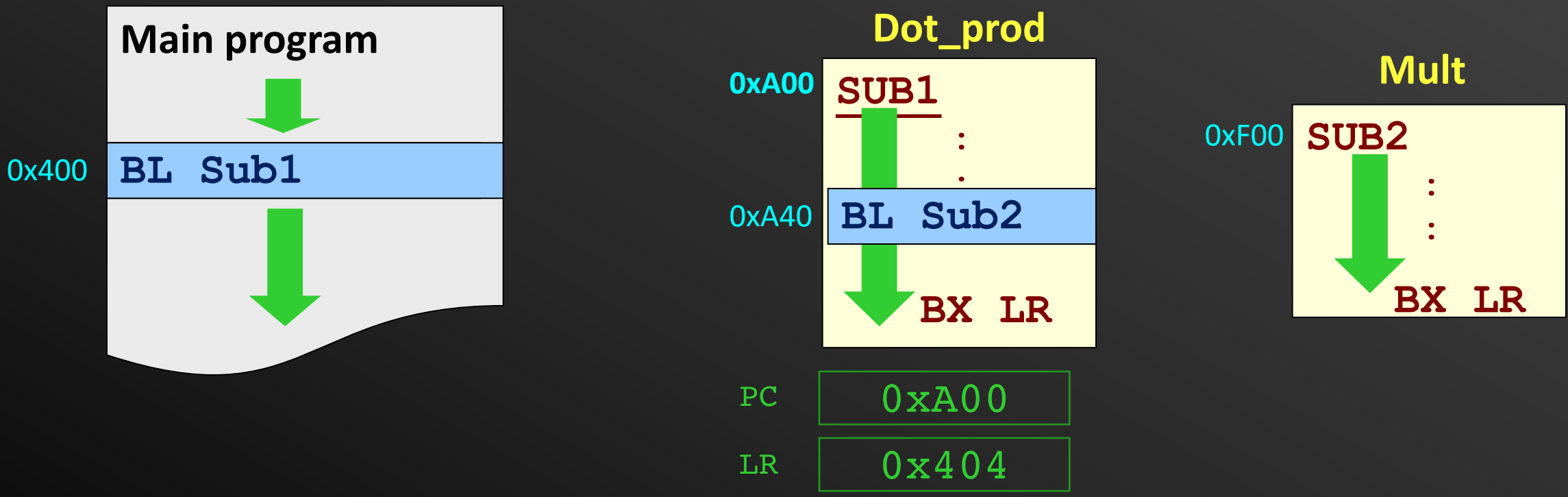
Example: Dot product of two arrays

- A subroutine will call another subroutine



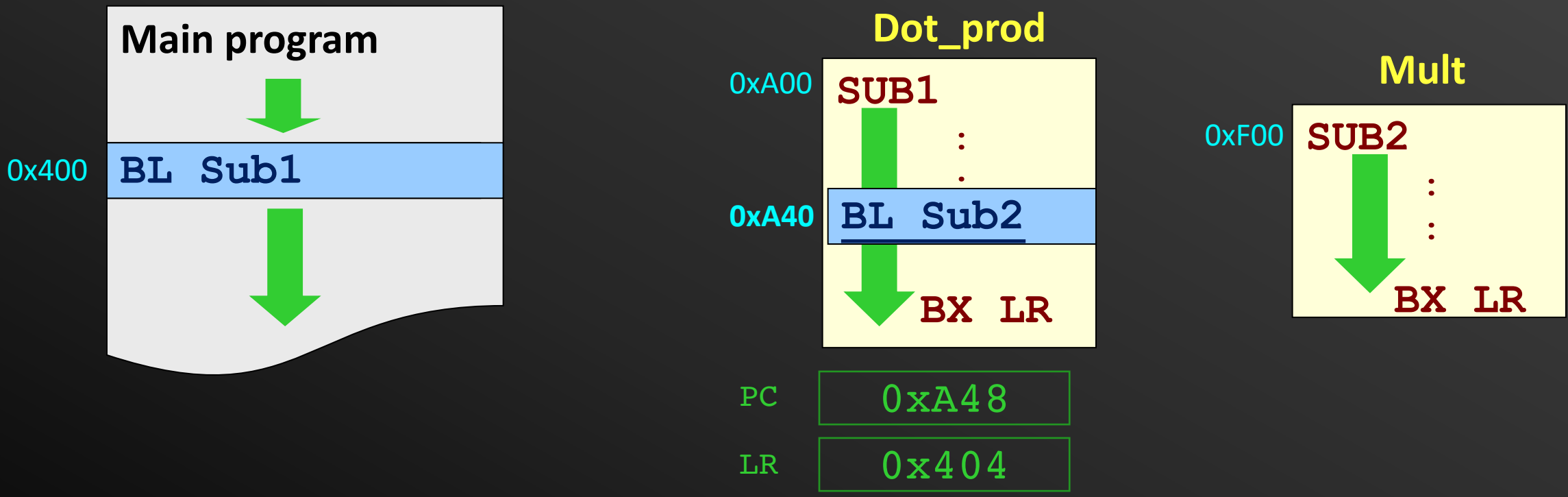
Example: Dot product of two arrays

- A subroutine will call another subroutine



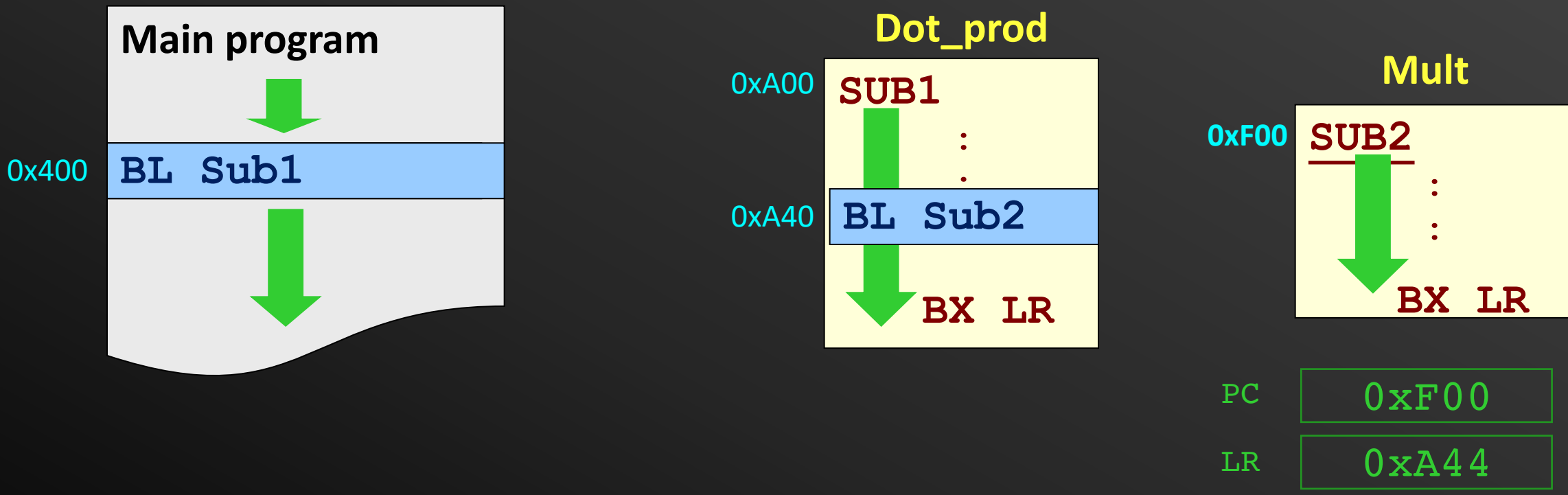
Example: Dot product of two arrays

- A subroutine will call another subroutine



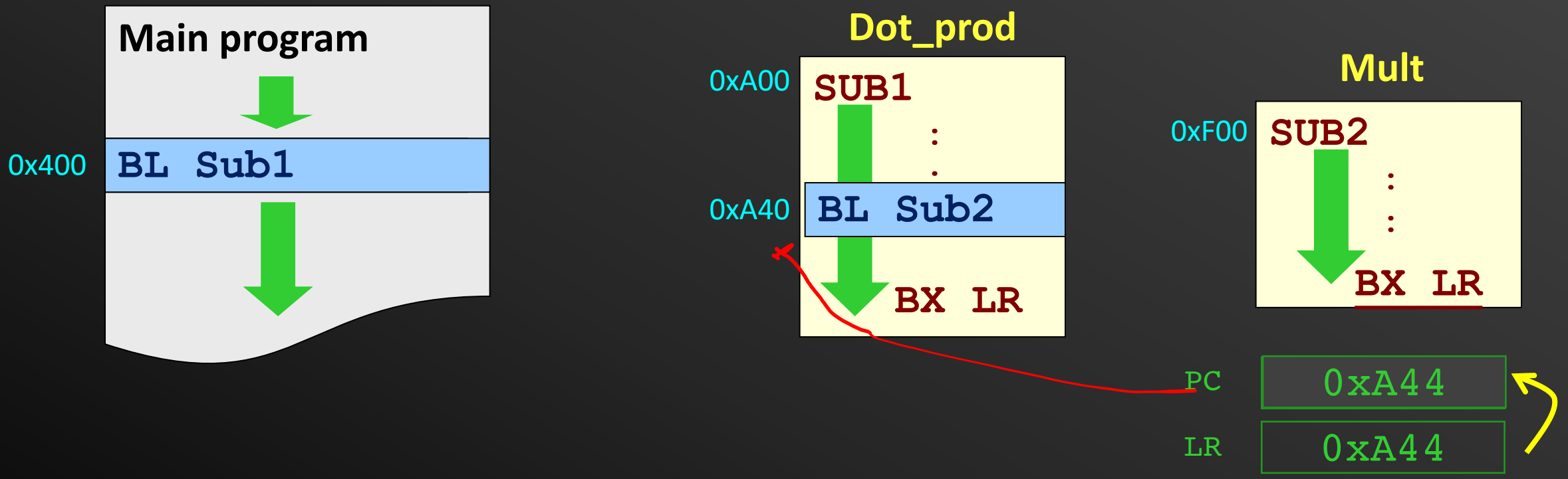
Example: Dot product of two arrays

- A subroutine will call another subroutine



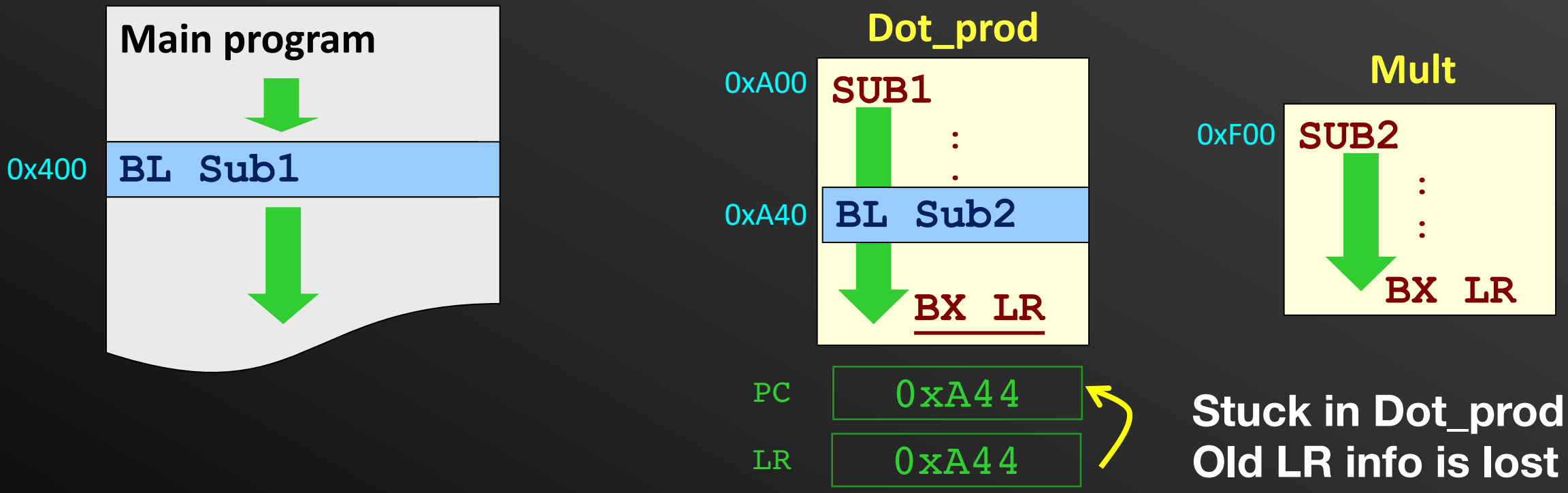
Example: Dot product of two arrays

- A subroutine will call another subroutine



Example: Dot product of two arrays

- A subroutine will call another subroutine



Support for nested Subroutine

- LR needs to be saved somewhere safe → **System stack**

- When so save it? Two options:
  - Just before any BL instruction
  - At the beginning of each subroutine

Example: Dot-product

- Write a subroutine that calculates the dot product of two vectors of **positive integers** using this equation:

$$Z = \sum_{i=0}^{N-1} X_i \times Y_i$$

- Bass addresses of X (0x100), Y(0x200), the array size N(20) and Z(0x300) are passed through the stack

- The subroutine will call another subroutine to calculate the product of two numbers:
  - Numbers are passed in R0,R1, return value in R12

Multiplication subroutine

```
Mult    MOV     R12,#0          ;Clear R12
```

Start by labeling the subroutine and placing the return instruction
Clearing (R2)

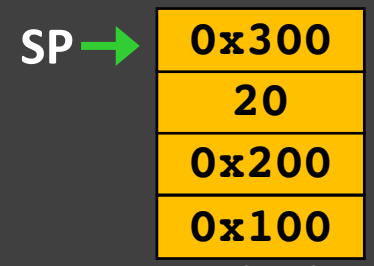
```
        MOV     PC, LR          ; same as bx lr
```

Multiplication subroutine

Iteratively add R0 to R12 for R1 times

```
Mult    MOV     R12,#0          ;Clear R12
Loop    ADD     R12,R12,R0      ;Add R0 to R12
        SUBS    R1,R1,#1        ;Decrement R1 with 1
        BNE     Loop
        MOV     PC, LR          ; same as bx lr
```

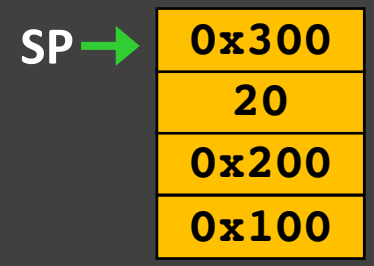
The Dot product Subroutine



System stack when entering DotProd

```
DotProd  STMFD    SP!,{R4-R7}  ;Store regs to stack
         LDR      R4 ,[SP,#28] ;Read location of X
         LDR      R5 ,[SP,#24] ;Read location of Y
         LDR      R6 ,[SP,#20] ;Read arrays length


         LDMFD    SP!,{R4-R7}  ; Restore registers
         MOV      PC, LR       ; same as bx lr
```

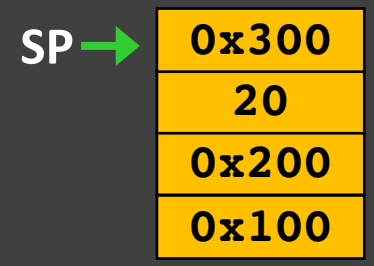
The Dot product Subroutine



System stack when entering DotProd

```
DotProd  STMFD    SP!,{R4-R7}  ;Store regs to stack
         LDR      R4 ,[SP,#28] ;Read location of X
         LDR      R5 ,[SP,#24] ;Read location of Y
         LDR      R6 ,[SP,#20] ;Read arrays length
         MOV      R7, #0       ; Clear R7
Loop1    LDR      R0 ,[R4],#4  ; Get X[i]
         LDR      R1 ,[R5],#4  ; Get Y[i]

         BL       Mult         ; Call Mult Subroutine

         LDMFD    SP!,{R4-R7}  ; Restore registers
         MOV      PC, LR       ; same as bx lr
```

The Dot product Subroutine

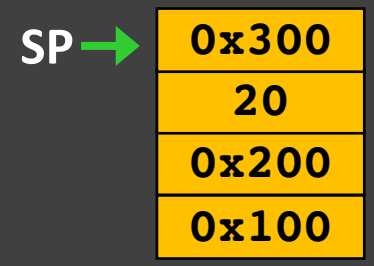


System stack when entering DotProd

```
DotProd     STMFD       SP!,{R4-R7}       ;Store regs to stack
            LDR         R4 ,[SP,#28]      ;Read location of X
            LDR         R5 ,[SP,#24]      ;Read location of Y
            LDR         R6 ,[SP,#20]      ;Read arrays length
            MOV         R7, #0            ; Clear R7 (Sum)
Loop1       LDR         R0 ,[R4],#4       ; Get X[i]
            LDR         R1 ,[R5],#4       ; Get Y[i]

            BL          Mult

            ADD         R7,R7,R12

            LDMFD       SP!,{R4-R7}       ; Restore registers
            MOV         PC, LR            ; same as bx lr
```

```
Mult        MOV         R12,#0            ;Clear R12
Loop        ADD         R12,R12,R0        ;Add R0 to R12
            SUBS        R1,R1,#1          ;Decrement R1 with 1
            BNE         Loop
            MOV         PC, LR            ; same as bx lr
```

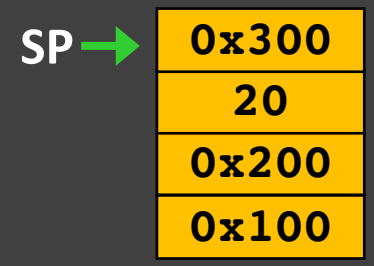
The Dot product Subroutine



System stack when entering DotProd

```
DotProd     STMFD       SP!,{R4-R7}       ;Store regs to stack
            LDR         R4 ,[SP,#28]      ;Read location of X
            LDR         R5 ,[SP,#24]      ;Read location of Y
            LDR         R6 ,[SP,#20]      ;Read arrays length
            MOV         R7, #0            ; Clear R7 (Sum)
Loop1       LDR         R0 ,[R4],#4       ; Get X[i]
            LDR         R1 ,[R5],#4       ; Get Y[i]

            BL          Mult              ; Call Mult Subroutine

            ADD         R7,R7,R12         ; Add the product to R7
            SUBS        R6,R6,#1          ; Reduce the counter by 1
            BNE         Loop1             ; not 0 then repeat

            LDMFD       SP!,{R4-R7}       ; Restore registers
            MOV         PC, LR            ; same as bx lr
```

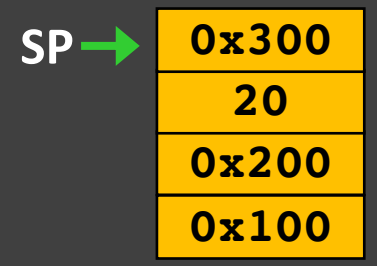
The Dot product Subroutine



System stack when entering DotProd

```
DotProd     STMFD       SP!,{R4-R7}         ;Store regs to stack
            LDR         R4 ,[SP,#28]              ;Read location of X
            LDR         R5 ,[SP,#24]              ;Read location of Y
            LDR         R6 ,[SP,#20]              ;Read arrays length
            MOV         R7, #0                    ; Clear R7 (Sum)
Loop1       LDR         R0 ,[R4],#4               ; Get X[i]
            LDR         R1 ,[R5],#4         ; Get Y[i]

            BL          Mult                ; Call Mult Subroutine

            ADD         R7,R7,R12           ; Add the product to R7
            SUBS        R6,R6,#1            ; Reduce the counter by 1
            BNE         Loop1               ; not 0 then repeat
            LDR         R4, [SP,#16]              ; read destination address
            STR         R7, [R4]            ; Store in destination address
            LDMFD       SP!,{R4-R7}         ; Restore registers
            MOV         PC, LR                    ; same as bx lr
```

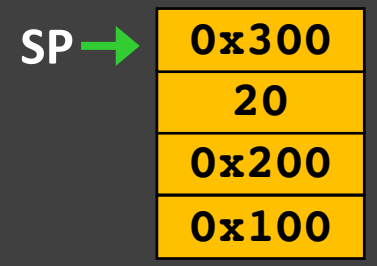
The Dot product Subroutine



System stack when entering DotProd

```
DotProd     STMFD       SP!,{R4-R7}       ;Store regs to stack
            LDR         R4 ,[SP,#28]      ;Read location of X
            LDR         R5 ,[SP,#24]      ;Read location of Y
            LDR         R6 ,[SP,#20]      ;Read arrays length
            MOV         R7, #0            ; Clear R7 (Sum)
Loop1       LDR         R0 ,[R4],#4       ; Get X[i]
            LDR         R1 ,[R5],#4       ; Get Y[i]
            STR         LR ,[SP,#-4]!     ; Push Link register to SP
            BL          Mult              ; Call Mult Subroutine
            LDR         LR ,[SP],#4       ; Pop Link Register from SP
            ADD         R7,R7,R12         ; Add the product to R7
            SUBS        R6,R6,#1          ; Reduce the counter by 1
            BNE         Loop1             ; not 0 then repeat
            LDR         R4, [SP,#16]      ; read destination address
            STR         R7, [R4]          ; Store in destination address
            LDMFD       SP!,{R4-R7}       ; Restore registers
            MOV         PC, LR            ; same as bx lr
```

The Dot product Subroutine



System stack when entering `DotProd`

✓ Straight forward, no other impact on the code

```
STR     LR ,[SP,#-4]!     ; Push Link register to SP
BL      Mult              ; Call Mult Subroutine
LDR     LR ,[SP],#4       ; Pop Link Register from SP
```

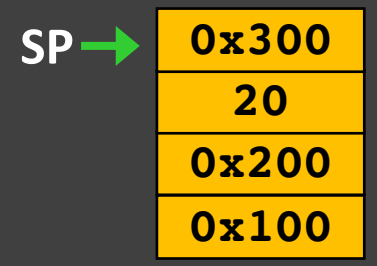
✗ Different instruction structure

Support for nested Subroutine

- LR needs to be saved somewhere safe → **System stack**

- When so save it? Two options:
  - Just before any BL instruction
  - At the beginning of each subroutine

The Dot product Subroutine



System stack when entering DotProd

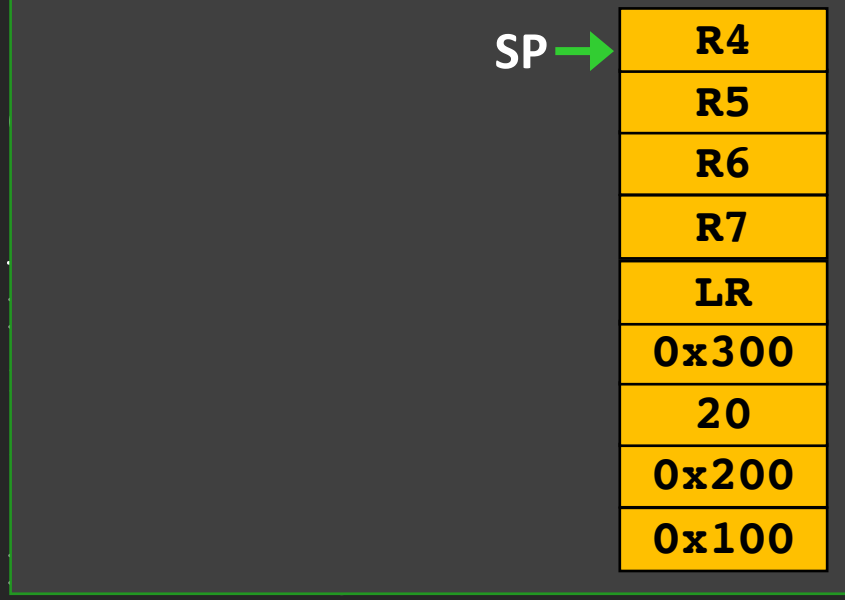
```
DotProd     STMFD       SP!,{R4-R7,LR}    ;Store regs to stack
            LDR         R4 ,[SP,#28]      ;Read location of X
            LDR         R5 ,[SP,#24]      ;Read location of Y
            LDR         R6 ,[SP,#20]      ;Read arrays length
            MOV         R7, #0            ; Clear R7 (Sum)
Loop1       LDR         R0 ,[R4],#4       ; Get X[i]
            LDR         R1 ,[R5],#4       ; Get Y[i]

            BL          Mult              ; Call Mult Subroutine

            ADD         R7,R7,R12         ; Add the product to R7
            SUBS        R6,R6,#1          ; Reduce the counter by 1
            BNE         Loop1             ; not 0 then repeat
            LDR         R4, [SP,#16]      ; read destination address
            STR         R7, [R4]          ; Store in destination address
            LDMFD       SP!,{R4-R7,LR}    ; Restore registers
            MOV         PC, LR            ; same as bx lr
```

The Dot product Subroutine

```
DotProd     STMFD       SP!,{R4-R7,LR}     ;Store regs to stack
            LDR         R4 ,[SP,#28]       ;R
            LDR         R5 ,[SP,#24]       ;R
            LDR         R6 ,[SP,#20]       ;R
            MOV         R7, #0             ;
Loop1       LDR         R0 ,[R4],#4        ;
            LDR         R1 ,[R5],#4        ;

            BL          Mult               ;

            ADD         R7,R7,R12          ;
            SUBS        R6,R6,#1           ;
            BNE         Loop1              ;
            LDR         R4, [SP,#16]       ;
            STR         R7, [R4]           ;                                    ss
            LDMFD       SP!,{R4-R7,LR}     ;
            MOV         PC, LR             ; same as bx lr
```

System stack when entering DotProd

We now push 5 registers to stack
Need to update the offsets

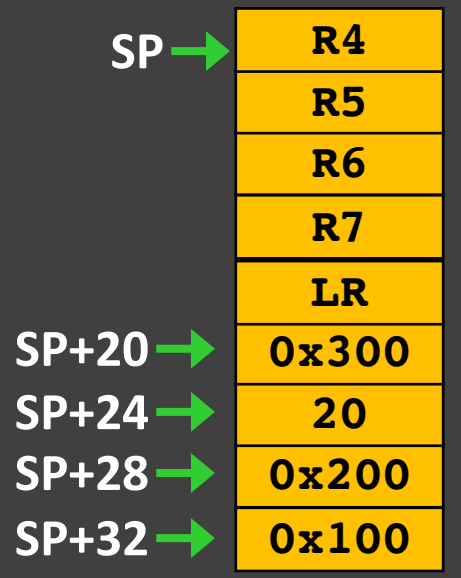


The Dot product Subroutine

```
DotProd     STMFD       SP!,{R4-R7,LR}     ;Store regs to stack
            LDR         R4 ,[SP,#32]       ;R
            LDR         R5 ,[SP,#28]       ;R
            LDR         R6 ,[SP,#24]       ;R
            MOV         R7, #0             ;
Loop1       LDR         R0 ,[R4],#4        ;
            LDR         R1 ,[R5],#4        ;

            BL          Mult               ;

            ADD         R7,R7,R12          ;
            SUBS        R6,R6,#1           ;
            BNE         Loop1              ;
            LDR         R4, [SP,#20]       ;
            STR         R7, [R4]           ;
            LDMFD       SP!,{R4-R7,LR}     ;
            MOV         PC, LR             ; same as bx lr
```

System stack when entering DotProd

We now push 5 registers to stack
Need to update the offsets



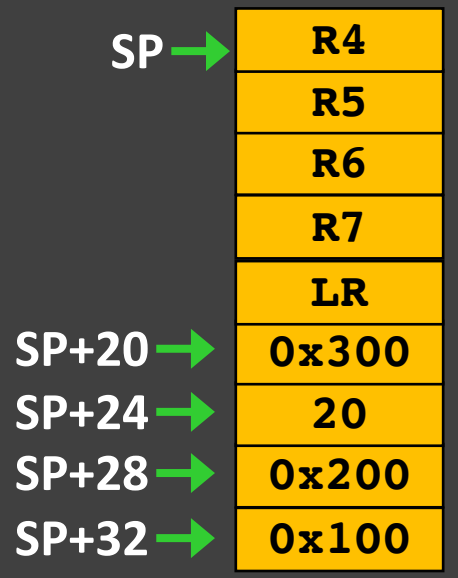
The Dot product Subroutine

```
DotProd     STMFD       SP!,{R4-R7,LR}      ;Store regs to stack
            LDR         R4 ,[SP,#32]        ;R
            LDR         R5 ,[SP,#28]        ;R
            LDR         R6 ,[SP,#24]        ;R
            MOV         R7, #0              ;
Loop1       LDR         R0 ,[R4],#4         ;
            LDR         R1 ,[R5],#4         ;

            BL          Mult                ;

            ADD         R7,R7,R12           ;
            SUBS        R6,R6,#1            ;
            BNE         Loop1               ;
            LDR         R4, [SP,#20]        ;
            STR         R7, [R4]            ;
            LDMFD       SP!,{R4-R7,PC}      ;
```

Why is this correct?

System stack when entering DotProd

We now push 5 registers to stack
Need to update the offsets



Recursive Subroutine

- A recursive routine calls itself within its own body.
- Recursion is an elegant way to solve algorithms or mathematical expressions that have **systematic repetitions**. (e.g. factorial **n! = n×(n-1)×(n-2)….2×1**)

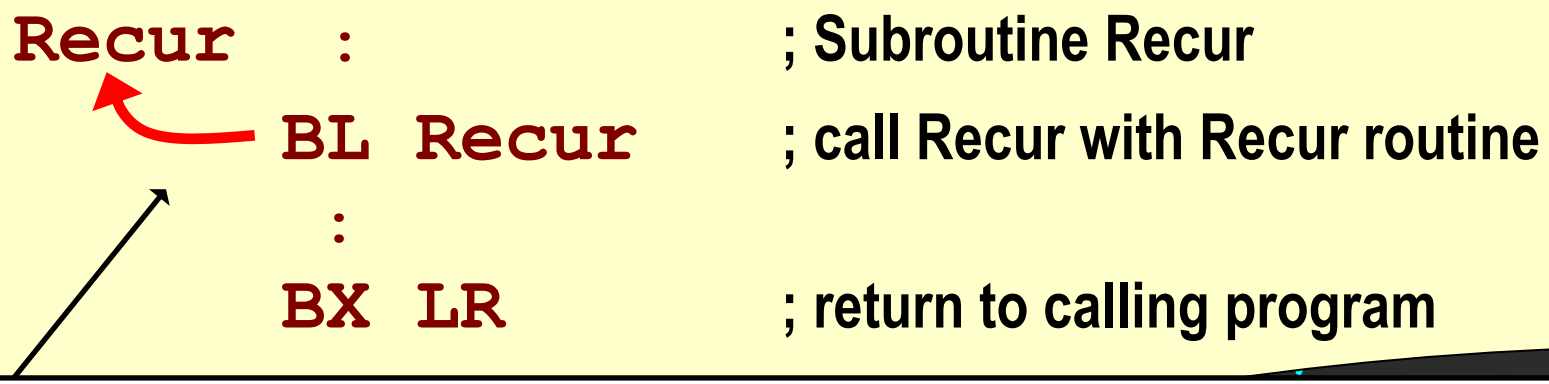
```
int factorial(int n){
    if (n<0)
     return -1; //sanity check
    if (n==0)
     return 1;
    else
     return n * factorial(n-1);
}
```

Recursive Subroutine

- A recursive routine calls itself within its own body.
- Recursion is an elegant way to solve algorithms or mathematical expressions that have **systematic repetitions**. (e.g. factorial **n! = n×(n-1)×(n-2)….2×1**)

Main problem: Without saving LR, execution will be stuck

Recursive Code Example



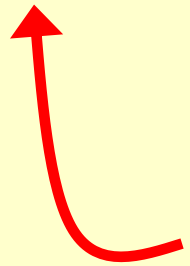
```
Recur   :                ; Subroutine Recur
        BL  Recur        ; call Recur with Recur routine
        :
        BX  LR           ; return to calling program
```

Another problem: Some condition must be reached that allows `BL Recur` to be skipped in order to avoid infinite recursion.

Recursive Subroutine

- A recursive routine calls itself within its own body.
- Recursion is an elegant way to solve algorithms or mathematical expressions that have **systematic repetitions**. (e.g. factorial **n! = n×(n-1)×(n-2)….2×1**)

Recursive Code Example



```
Recur   STMFD SP!,{…,LR}        ; Subroutine Recur
```

Condition to terminate the subroutine **(branch to Done)**

```
        :
        BL Recur                ; call Recur with Recur routine
        :
Done    LDMFD SP!,{…,LR}
        BX LR                   ; return to calling program
```

Example: Fibonacci Sequence

- Number sequence

  0,1,1,2,3,5,8,13,21,34,55

- Starting from index 1

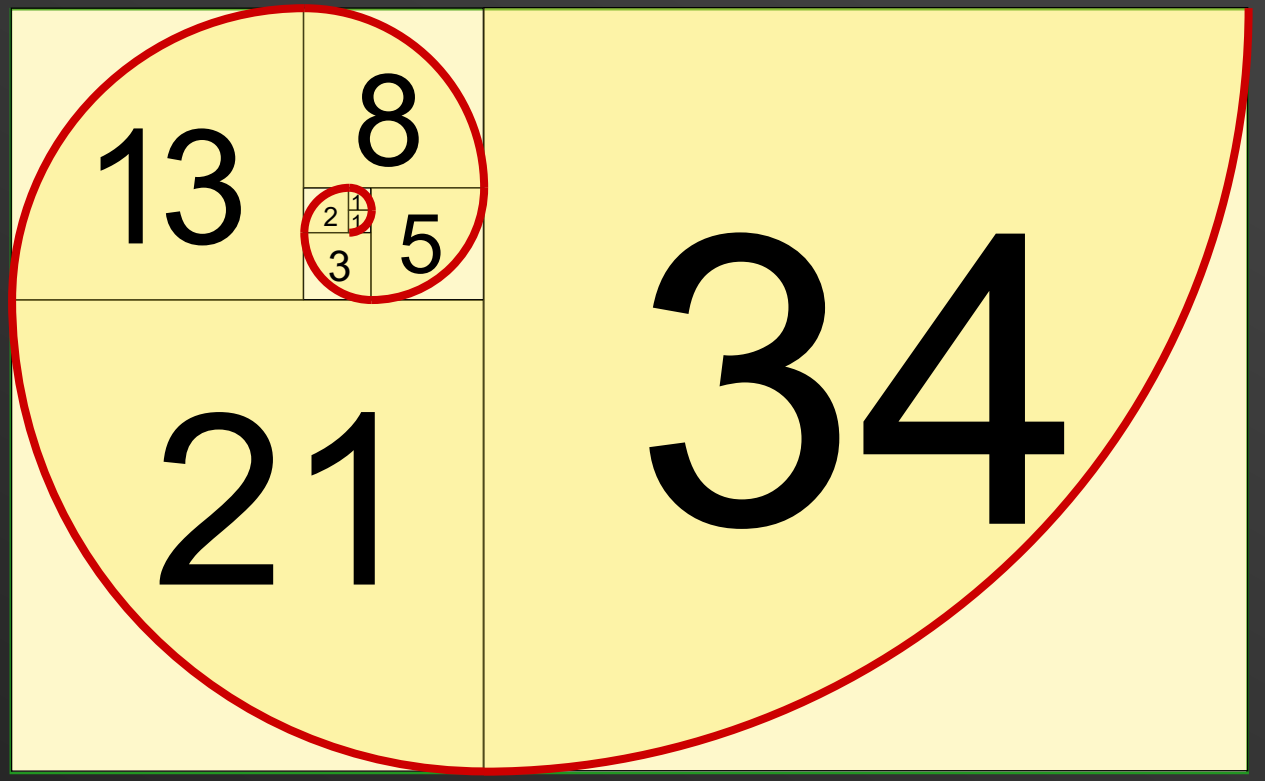
  $F(n) = F(n-1) + F(n-2)$

  $F(2) = 1$

  $F(1) = 0$

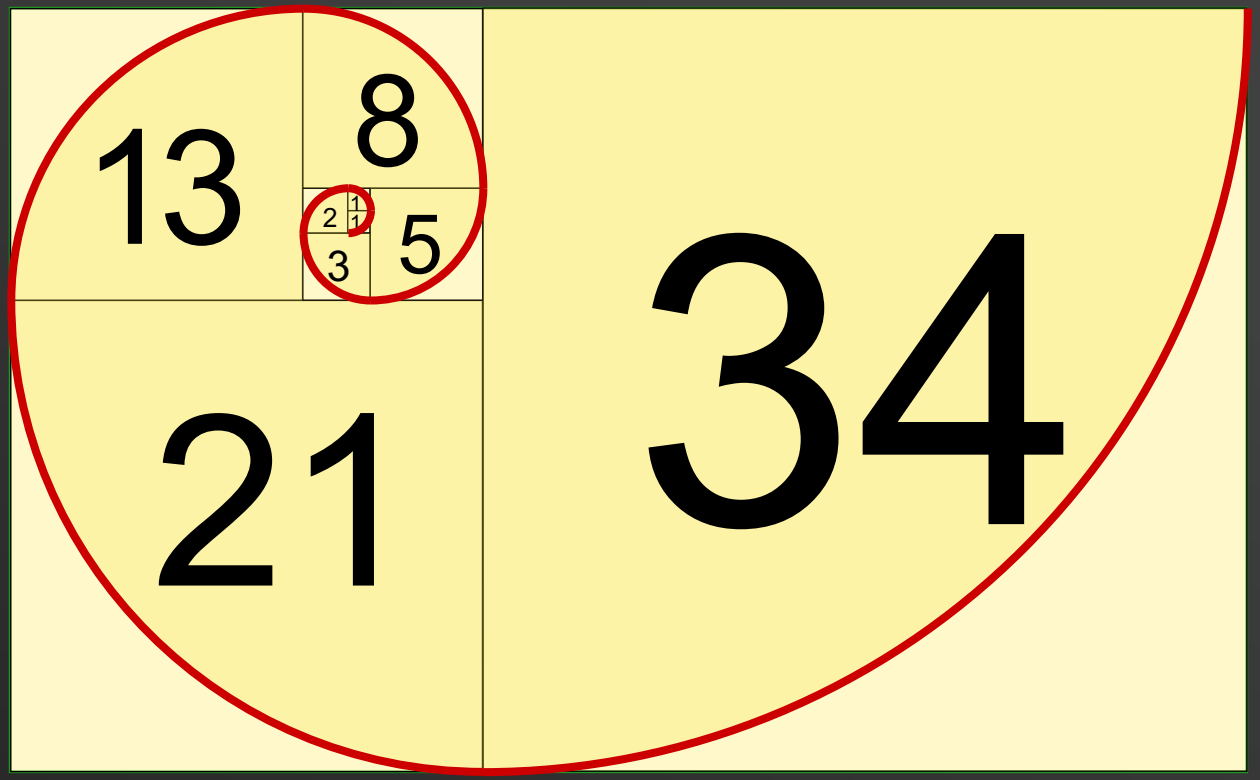
  Pass parameter in stack

  Return result in R12



Example: Fibonacci Sequence

- Number sequence
  0,1,1,2,3,5,8,13,21,34,55




```
int Fibonacci(int n){
     int result;
    if (n==1)
     result=0;
    elseif (n==2)
     result=1;
    else
     result = Fibonacci(n-1) + Fibonacci(n-2);
    return result;
}
```

Example Solution

System stack when entering `Fib`

```
Fib             STMFD           SP!,{R4-R6,LR}      ;Store regs to stack
```

```
Done            LDMFD           SP!,{R4-R6,LR}      ; Restore registers
                MOV             PC, LR              ; same as bx lr
```

Example Solution

System stack when entering `Fib`

```
Fib     STMFD   SP!,{R4-R6,LR}   ;Store regs to stack
        LDR     R4 ,[SP,#16]     ;Read Value of n
        CMP     R4 ,#1           ;Compare with 1 (if n==1)
        MOVEQ   R12,#0           ;Assign result with 0
        CMP     R4 ,#2           ;Compare with 2 (if n==2)
        MOVEQ   R12,#1           ; Assign result with 1
        BLE     Done             ; if n less than equal 2 finish

Done    LDMFD   SP!,{R4-R6,LR}   ; Restore registers
        MOV     PC, LR           ; same as bx lr
```

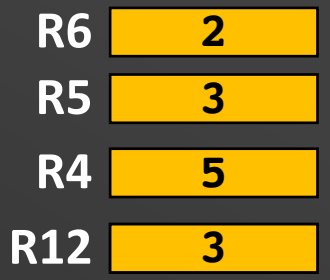
Example Solution

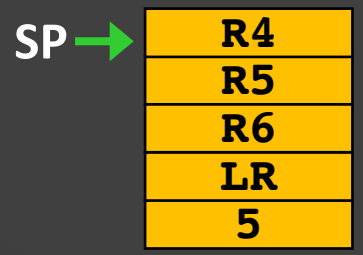

```
Fib     STMFD   SP!,{R4-R6,LR}   ;Store regs to stack
        LDR     R4 ,[SP,#16]     ;Read Value of n
        CMP     R4 ,#1           ;Compare with 1 (if n==1)
        MOVEQ   R12,#0           ;Assign result with 0
        CMP     R4 ,#2           ;Compare with 2 (if n==2)
        MOVEQ   R12,#1           ; Assign result with 1
        BLE     Done             ; if n less than equal 2 finish
        SUB     R5,R4,#1         ; Get n-1
        STMFD   SP!,{R5}         ;Push n-1 to stack
        BL      Fib              ; calculate Fib(n-1)
        LDMFD   SP!,{R5}         ; Pop from stack

Done    LDMFD   SP!,{R4-R6,LR}   ; Restore registers
        MOV     PC, LR           ; same as bx lr
```

Example Solution


```
Fib     STMFD   SP!,{R4-R6,LR}   ;Store regs to stack
        LDR     R4 ,[SP,#16]     ;Read Value of n
        CMP     R4 ,#1           ;Compare with 1 (if n==1)
        MOVEQ   R12,#0           ;Assign result with 0
        CMP     R4 ,#2           ;Compare with 2 (if n==2)
        MOVEQ   R12,#1           ; Assign result with 1
        BLE     Done             ; if n less than equal 2 finish
        SUB     R5,R4,#1         ; Get n-1
        STMFD   SP!,{R5}         ;Push n-1 to stack
        BL      Fib              ; calculate Fib(n-1)
        LDMFD   SP!,{R5}         ; Pop from stack
        MOV     R6,R12           ;Save result in temp register
        SUB     R5,R4,#2         ; Get n-2
        STMFD   SP!,{R5}         ;Push n-2 to stack
        BL      Fib              ; calculate Fib (n-2)
        LDMFD   SP!,{R5}         ; Pop from stack

Done    LDMFD   SP!,{R4-R6,LR}   ; Restore registers
        MOV     PC, LR           ; same as bx lr
```

Example Solution






```
Fib     STMFD   SP!,{R4-R6,LR}   ;Store regs to stack
        LDR     R4 ,[SP,#16]     ;Read Value of n
        CMP     R4 ,#1           ;Compare with 1 (if n==1)
        MOVEQ   R12,#0           ;Assign result with 0
        CMP     R4 ,#2           ;Compare with 2 (if n==2)
        MOVEQ   R12,#1           ; Assign result with 1
        BLE     Done             ; if n less than equal 2 finish
        SUB     R5,R4,#1         ; Get n-1
        STMFD   SP!,{R5}         ;Push n-1 to stack
        BL      Fib              ; calculate Fib(n-1)
        LDMFD   SP!,{R5}         ; Pop from stack
        MOV     R6,R12           ;Save result in temp register
        SUB     R5,R4,#2         ; Get n-2
        STMFD   SP!,{R5}         ;Push n-2 to stack
        BL      Fib              ; calculate Fib (n-2)
        LDMFD   SP!,{R5}         ; Pop from stack
        ADD     R12,R12,R6       ; Calculate Fib(n-1) + Fib(n-2)
Done    LDMFD   SP!,{R4-R6,LR}   ; Restore registers
        MOV     PC, LR           ; same as bx lr
```

Example Solution

```
Fib         STMFD       SP!,{R4-R6,LR}      ;Store regs to stack
            LDR         R4 ,[SP,#16]        ;Read Value of n
            CMP         R4 ,#1              ;Compare with 1 (if n==1)
            MOVEQ       R12,#0              ;Assign result with 0
            CMP         R4 ,#2              ;Compare with 2 (if n==2)
            MOVEQ       R12,#1              ; Assign result with 1
            BLE         Done                ; if n less than equal 2 finish
            SUB         R5,R4,#1            ; Get n-1
            STMFD       SP!,{R5}            ;Push n-1 to stack
            BL          Fib                 ; calculate Fib(n-1)
            LDMFD       SP!,{R5}            ; Pop from stack
            MOV         R6,R12              ;Save result in temp register
            SUB         R5,R4,#2            ; Get n-2
            STMFD       SP!,{R5}            ;Push n-2 to stack
            BL          Fib                 ; calculate Fib (n-2)
            LDMFD       SP!,{R5}            ; Pop from stack
            ADD         R12,R12,R6          ; Calculate Fib(n-1) + Fib(n-2)
Done        LDMFD       SP!,{R4-R6,LR}      ; Restore registers
            MOV         PC, LR              ; same as bx lr
```

Summary

- Nested subroutines are key for truly modular designs
  - Need to ensure that return address is not overwritten.
- Recursive subroutine is very efficient implementation of subroutines
  - Ensure: 1) stopping condition and 2) return address stored